Random Data Generation

Machine learning models start out "blank" and need initial random guesses to begin learning. The np.random module builds massive grids filled with random numbers or specific statistical distributions instantly so your algorithms have a starting point.

```
import numpy as np

# Generates a 2x3 grid of random decimals between 0 and 1
starting_weights = np.random.rand(2, 3)
print(starting_weights)
```

```
[[0.11096127 0.41143224 0.64385813]
 [0.86691395 0.03599434 0.79121537]]
```

Boolean Masking (Conditional Filtering)

Instead of writing a slow for loop to inspect every single number, you ask NumPy a True/False question directly inside the brackets. It creates a digital "mask" over the array, instantly revealing only the numbers that pass the mathematical test.

```
data = np.array([50, 150, 200, 30])

# Instantly extracts only the numbers greater than 100
high_values = data[data > 100]
print(high_values) # Output: [150 200]
```

```
[150 200]
```

Stacking and Merging

Real-world data rarely comes in one perfect file. If you have historical records from 2025 and 2026, np.vstack() (vertical stack) glues them top-to-bottom into a master database, while np.hstack() (horizontal stack) glues new columns side-by-side.

```
january = np.array([1, 2, 3])
february = np.array([4, 5, 6])

# Glues the two arrays together into a 2-row grid.
# Note the double parentheses (()) required to group them.
combined_months = np.vstack((january, february))
print(combined_months)
```

```
[[1 2 3]
 [4 5 6]]
```

Finding and Replacing

The np.where() function is an instant find-and-replace tool for millions of data points. It requires exactly three instructions: the mathematical rule to check, the new replacement value if the rule is True, and the fallback value if the rule is False.

```
sensor_data = np.array([72, -1, 74, -1])

# Rule: If data is -1 (error), replace with 0. Otherwise, keep the original.
fixed_data = np.where(sensor_data == -1, 0, sensor_data)
print(fixed_data) # Output: [72  0 74  0]
```

```
[72  0 74  0]
```

Linear Algebra

A neural network makes predictions by multiplying your input data by its learned mathematical "weights." The np.dot() command calculates the dot product (multiplying matching positional elements across two arrays and adding the results together), which is the core engine of AI predictions.

```
inputs = np.array([2, 3])
weights = np.array([0.5, 1.0])

# Math: (2 * 0.5) + (3 * 1.0) = 1.0 + 3.0 = 4.0
ai_prediction = np.dot(inputs, weights)
print(ai_prediction) # Output: 4.0
```

```
4.0
```